

AEM + Frontend Interview Handbook 2024–2026

This handbook converts your interview notes into real interviewer-style questions with short answers, examples, and company repetition mapping so you can revise fast before interviews.

1. JavaScript

1.1 What is the difference between `==` and `===`?

Short answer: `==` checks value after type conversion, while `===` checks both value and type without conversion.

Example:

```
'5' == 5    // true
'5' === 5   // false
```

Asked by: Embitel consultancy, Wipro, repeated in general JS rounds.

1.2 What is the difference between `let`, `const`, and `var`?

Short answer: `var` is function-scoped and can be redeclared, `let` is block-scoped and can be reassigned, `const` is block-scoped and cannot be reassigned.

Example:

```
if (true) {
  var a = 1;
  let b = 2;
  const c = 3;
}
console.log(a); // 1
// console.log(b); // error
// console.log(c); // error
```

Asked by: Embitel consultancy, Techzuf, Wipro, repeated in many frontend rounds.

1.3 What is the difference between `null` and `undefined`?

Short answer: `undefined` means a value has not been assigned yet. `null` means the value is intentionally empty.

Example:

```
let a;  
console.log(a); // undefined  
  
let b = null;  
console.log(b); // null
```

Asked by: Embitel consultancy, repeated in frontend interviews.

1.4 What are closures?

Short answer: A closure happens when an inner function remembers variables from its outer function even after the outer function finishes.

Example:

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    return count;  
  };  
}  
  
const counter = outer();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

Asked by: Embitel consultancy, Concentrix, Deloitte, Publicis Sapient, Wipro.

1.5 What is lexical scope in closure?

Short answer: Lexical scope means a function can access variables from the scope where it was created.

Example:

```
function parent() {  
  let name = 'Amit';  
  function child() {  
    console.log(name);  
  }  
  child();  
}
```

Asked by: Publicis Sapient, Wipro.

1.6 What is hoisting?

Short answer: Hoisting means declarations are moved to the top of their scope before execution. Function declarations are fully hoisted, while `var` is hoisted with `undefined`.

Example:

```
console.log(a); // undefined
var a = 10;
```

Asked by: Deloitte, Accenture, repeated in JS rounds.

1.7 What is a callback function, and what is callback hell?

Short answer: A callback is a function passed into another function. Callback hell happens when many nested callbacks make code hard to read.

Example:

```
setTimeout(() => {
  console.log('Step 1');
  setTimeout(() => {
    console.log('Step 2');
  }, 1000);
}, 1000);
```

Asked by: Techzuf, Wipro, Accenture.

1.8 What are Promises and what are their states?

Short answer: A Promise handles asynchronous operations. It has three states: pending, fulfilled, and rejected.

Example:

```
const p = new Promise((resolve, reject) => {
  resolve('Done');
});
p.then(res => console.log(res));
```

Asked by: Techzuf, Fynd, Concentrix, Wipro.

1.9 Can a Promise be created with a normal function?

Short answer: Yes. A Promise is created using `new Promise(function(resolve, reject) {})`.

Example:

```
const p = new Promise(function(resolve, reject) {
  resolve('success');
```

```
});
```

Asked by: Techzuf.

1.10 What is `Promise.allSettled()`?

Short answer: It waits for all promises to finish and returns the status of each one, whether fulfilled or rejected.

Example:

```
Promise.allSettled([
  Promise.resolve('ok'),
  Promise.reject('error')
]).then(console.log);
```

Asked by: Fynd.

1.11 How does `async/await` work?

Short answer: `async` makes a function return a `Promise`. `await` pauses the function until the `Promise` resolves or rejects.

Example:

```
async function getData() {
  const res = await fetch('/api');
  const data = await res.json();
  console.log(data);
}
```

Asked by: Techzuf, Wipro.

1.12 What is the event loop?

Short answer: The event loop manages how JavaScript handles synchronous code, async tasks, the call stack, and callback queue.

Example:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
console.log('3');
// Output: 1 3 2
```

Asked by: Wipro, EY, Concentrix.

1.13 What is event bubbling?

Short answer: Event bubbling means an event starts from the target element and bubbles up to parent elements.

Example:

```
document.getElementById('child').addEventListener('click', () => console.log('child'))
document.getElementById('parent').addEventListener('click', () => console.log('parent'))
```

Asked by: EPAM.

1.14 What are call, apply, and bind?

Short answer: They are used to control the this value in functions. call and apply invoke immediately, while bind returns a new function.

Example:

```
function greet(city) {
  console.log(this.name + ' ' + city);
}
const user = { name: 'Amit' };
greet.call(user, 'Pune');
```

Asked by: SCSS round, repeated in JS interviews.

1.15 What is the difference between spread and rest operators?

Short answer: Spread expands values, rest collects values.

Example:

```
const arr1 = [1, 2];
const arr2 = [...arr1, 3];

function sum(...nums) {
  return nums.reduce((a, b) => a + b, 0);
}
```

Asked by: Fynd.

1.16 What is shallow copy vs deep copy?

Short answer: A shallow copy copies only the first level. A deep copy creates independent copies of nested objects too.

Example:

```
const obj = { a: 1, b: { c: 2 } };  
const shallow = { ...obj };
```

Asked by: Fynd.

1.17 What is strict mode in JavaScript?

Short answer: Strict mode makes JS safer by preventing silent errors and disallowing unsafe actions.

Example:

```
'use strict';  
x = 10; // error
```

Asked by: SCSS round.

1.18 What is the difference between polyfill and prototype?

Short answer: A prototype is the mechanism through which objects inherit methods. A polyfill is custom code added to support missing browser features.

Example:

```
if (!Array.prototype.includes) {  
  Array.prototype.includes = function(value) {  
    return this.indexOf(value) !== -1;  
  };  
}
```

Asked by: Wipro.

1.19 What are Web Workers, Service Workers, and Edge Workers?

Short answer: Web Workers run JS in a background thread, Service Workers handle caching/offline support, and Edge Workers run code closer to the CDN edge.

Asked by: Wipro.

1.20 What is localStorage, sessionStorage, cookies, and IndexedDB?

Short answer: localStorage stores long-term key-value data, sessionStorage stores per-tab session data, cookies are small browser/server data pieces, and IndexedDB is a browser database for larger structured data.

Asked by: Itvedant, SCSS round.

2. React

2.1 What are React Hooks?

Short answer: Hooks let function components use state and lifecycle features.

Example:

```
const [count, setCount] = useState(0);
```

Asked by: Embitel consultancy, HCL, NTT Data, repeated frequently.

2.2 Explain `useState` and `useEffect`.

Short answer: `useState` stores component state. `useEffect` handles side effects like API calls or subscriptions.

Example:

```
const [count, setCount] = useState(0);
useEffect(() => {
  document.title = `Count ${count}`;
}, [count]);
```

Asked by: Embitel consultancy, Truemed, Fynd.

2.3 What is `useEffect`, and how do you manage the dependency array?

Short answer: `useEffect` runs after render. The dependency array controls when it runs.

Example:

```
useEffect(() => {
  fetchData();
}, []); // runs once on mount
```

Asked by: Fynd, Embitel consultancy.

2.4 What are `useMemo`, `useCallback`, and `React.memo`?

Short answer: They help optimize performance by preventing unnecessary calculations and re-renders.

Example:

```
const memoizedValue = useMemo(() => heavyCalc(data), [data]);
const memoizedFn = useCallback(() => doSomething(id), [id]);
export default React.memo(MyComponent);
```

Asked by: Bajaj, Publicis Sapient, HCL.

2.5 What is `useReducer`?

Short answer: `useReducer` manages complex state changes using actions and a reducer function.

Example:

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    default:
      return state;
  }
}
```

Asked by: SCSS round.

2.6 Difference between Props and State.

Short answer: Props are passed from parent to child and are read-only. State is managed inside the component and can change.

Asked by: Embitel consultancy, HCL.

2.7 How do you pass values from child to parent components?

Short answer: Pass a callback function from parent to child, and call it from the child.

Example:

```
function Parent() {
  const handleData = value => console.log(value);
  return <Child sendData={handleData} />;
}
function Child({ sendData }) {
  return <button onClick={() => sendData('hello')}>Send</button>;
}
```

Asked by: Embitel consultancy.

2.8 What is a Higher-Order Component (HOC)?

Short answer: A HOC is a function that takes a component and returns a new enhanced component.

Example:


```
function withLoader(Component) {  
  return function Wrapped(props) {  
    return props.loading ? <p>Loading...</p> : <Component {...props} />;  
  };  
}
```

Asked by: Embitel consultancy, Wipro, practical asked in Bajaj notes.

2.9 Difference between HOC and a callback function.

Short answer: A HOC enhances components. A callback is a function passed to another function/component to execute later.

Asked by: Wipro.

2.10 What is prop drilling, and how does Context API solve it?

Short answer: Prop drilling means passing data through many layers. Context API lets deeply nested components access shared data directly.

Asked by: SCSS round.

2.11 Redux vs Context API — when would you choose Redux?

Short answer: Use Context for simple global data like theme or user info. Use Redux for large apps with complex state, predictable updates, middleware, and debugging.

Asked by: Truemedics, HCL, Sushant Bajaj notes.

2.12 What is Redux middleware? Role of Redux-Saga / Redux-Thunk?

Short answer: Middleware handles logic between dispatch and reducer. Thunk is good for simple async actions. Saga is useful for complex async flows.

Asked by: Truemedics, Publicis Sapient.

2.13 What is a side effect in Redux?

Short answer: A side effect is any async or external operation like API calls, timers, or logging. It should not be done in reducers.

Asked by: Publicis Sapient.

2.14 React lifecycle methods — class components vs function components.

Short answer: Class components use methods like `componentDidMount`; function components use `useEffect`.

Asked by: Truemedics, Techzuf, Fynd.

2.15 Difference between class components and function components.

Short answer: Class components use lifecycle methods and `this`; function components use hooks and are simpler.

Asked by: Bajaj.

2.16 What is the Virtual DOM, and what is Reconciliation?

Short answer: Virtual DOM is a JS copy of the real DOM. Reconciliation is React comparing old and new virtual DOM to update the real DOM efficiently.

Asked by: HCL.

2.17 Why are keys important in React lists?

Short answer: Keys help React identify which items changed, added, or removed.

Asked by: HCL.

2.18 What are controlled vs uncontrolled components?

Short answer: Controlled components are managed by React state. Uncontrolled components use the DOM directly.

Example:

```
const [value, setValue] = useState('');  
<input value={value} onChange={e => setValue(e.target.value)} />
```

Asked by: Publicis Sapient.

2.19 What are React chunks / code splitting?

Short answer: Code splitting breaks the app into smaller bundles that load when needed.

Asked by: NTT Data.

2.20 How does React Router work?

Short answer: React Router maps URLs to components in a single-page application.

Asked by: NTT Data, SCSS round practical.

2.21 How is React tested with Jest?

Short answer: Jest is used to write and run unit tests for React components and functions.

Asked by: HCL.

2.22 How do you manage state in a React application overall?

Short answer: Use local state for simple UI data, Context for shared lightweight state, and Redux for complex application-wide state.

Asked by: HCL.

2.23 How would you optimize a React app for performance?

Short answer: Use memoization, code splitting, lazy loading, optimized rendering, proper keys, caching, and avoid unnecessary re-renders.

Asked by: Publicis Sapient, Sushant Bajaj notes.

2.24 Practical: build a controlled input that displays typed text.

Example:

```
function App() {
  const [text, setText] = useState('');
  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <p>{text}</p>
    </div>
  );
}
```

Asked by: HCL.

2.25 Practical: build an Accordion component.

Example:

```
function Accordion() {
  const [open, setOpen] = useState(false);
  return (
    <div>
      <button onClick={() => setOpen(!open)}>Toggle</button>
      {open && <p>Content</p>}
    </div>
  );
}
```

Asked by: Fynd, NTT Data.

3. HTML, CSS, and SCSS

3.1 What is the difference between CSS and SCSS?

Short answer: SCSS is a CSS preprocessor that supports variables, mixins, nesting, and functions.

Asked by: Embitel consultancy, EPAM.

3.2 What is the transform property in CSS?

Short answer: It changes the position, size, rotation, or skew of an element.

Example:

```
.box {  
  transform: rotate(45deg) scale(1.2);  
}
```

Asked by: Embitel consultancy.

3.3 Difference between Flexbox and Grid.

Short answer: Flexbox is mainly one-dimensional, while Grid is two-dimensional.

Asked by: Embitel consultancy, Deloitte, EPAM, Wipro, Accenture, EY.

3.4 How do you center text over a full-width banner image?

Short answer: Use a wrapper with `position: relative`, set the image full width, and place text absolutely in the center.

Example:

```
<div class="banner">  
    
  <div class="content">Welcome</div>  
</div>
```

```
.banner { position: relative; }  
.banner img { width: 100%; display: block; }  
.content {  
  position: absolute;  
  top: 50%; left: 50%;  
  transform: translate(-50%, -50%);  
}
```

Asked by: EY.

3.5 You have 10 cards and need layout 4 + 4 + 2 using Flexbox. How do you do it?

Short answer: Use `display: flex, flex-wrap: wrap`, and fixed width like 25% for first rows, then adjust last row with the same basis or responsive gaps.

Example:

```
.container { display: flex; flex-wrap: wrap; gap: 16px; }  
.card { flex: 0 0 calc(25% - 16px); }
```

Asked by: EY.

3.6 How do you show one image for mobile and another for desktop using HTML only?

Short answer: Use the `<picture>` element with multiple `<source>` tags.

Example:

```
<picture>  
  <source media="(max-width: 767px)" srcset="mobile.jpg">  
  <source media="(min-width: 768px)" srcset="desktop.jpg">  
    
</picture>
```

Asked by: EY, SCSS round.

3.7 What is the difference between pseudo-class and pseudo-element?

Short answer: A pseudo-class targets an element state, while a pseudo-element targets a part of an element.

Example: `:hover` is pseudo-class, `::before` is pseudo-element.

Asked by: SCSS round, Wipro.

3.8 What is the difference between semantic and non-semantic tags?

Short answer: Semantic tags describe meaning, like `<header>` or `<article>`. Non-semantic tags are used for layout and styling.

Asked by: Deloitte, Wipro.

4. AEM

4.1 What is AEM?

Short answer: Adobe Experience Manager is a CMS used to manage websites, digital assets, forms, and headless content.

Asked by: Embitel consultancy, Morgan Stanley, HCL, TCS.

4.2 Explain AEM architecture.

Short answer: AEM mainly works with Author, Publish, and Dispatcher. Authors create content in Author, content is published to Publish, and Dispatcher caches and filters requests before serving users.

Asked by: Bajaj, Sushant Bajaj notes, Morgan Stanley, Wipro, 3 Nov Bajaj.

4.3 What is Sling in AEM?

Short answer: Sling is the web framework in AEM that maps requests to resources and scripts.

Asked by: Embitel consultancy, Sushant Bajaj notes, 3 Nov Bajaj.

4.4 What is JCR?

Short answer: JCR is the Java Content Repository where AEM stores content as nodes and properties.

Asked by: Capgemini, 3 Nov Bajaj.

4.5 What is a Sling Model?

Short answer: A Sling Model is a Java class used to map AEM resource data into reusable backend objects.

Example:

```
@Model(adaptables = Resource.class)
public class BannerModel {
    @ValueMapValue
    private String title;

    public String getTitle() {
        return title;
    }
}
```

Asked by: Embitel consultancy, IndusInd Bank, EY.

4.6 What is @ValueMapValue and what other annotations are common in Sling Models?

Short answer: @ValueMapValue injects dialog property values from the resource. Other common annotations are @Inject, @Self, @OSGiService, @SlingObject, and @Via.

Asked by: EY, Capgemini, IndusInd Bank.

4.7 How do you consume Sling Model data in HTL/Sightly?

Short answer: Use data-sly-use to load the model and then access getters in HTL.

Example:

```
<div data-sly-use.banner="com.project.core.models.BannerModel">
  <h1>${banner.title}</h1>
</div>
```

Asked by: EY, 3 Nov Bajaj.

4.8 What is the difference between data-sly-list and data-sly-repeat?

Short answer: Both iterate collections. data-sly-list gives loop helper objects and is more flexible; data-sly-repeat repeats the element directly.

Asked by: EPAM, EY.

4.9 What is a Servlet?

Short answer: A Servlet is a Java class that handles HTTP requests and responses in AEM.

Asked by: IBM, EY, IndusInd Bank.

4.10 What are clientlibs in AEM?

Short answer: Client libraries manage CSS and JS in AEM using categories, dependencies, and embed.

Asked by: Deloitte, Capgemini, EPAM, EY, TCS.

4.11 Difference between embed and dependencies in clientlibs.

Short answer: dependencies loads another clientlib separately. embed merges another clientlib into the current one.

Asked by: Bajaj, Capgemini, EPAM, EY.

4.12 If vendor CSS contains Bootstrap, should you use embed or dependencies?

Short answer: Usually use dependencies if you want it loaded separately and reused. Use embed when you want one combined output.

Asked by: EY.

4.13 What is clientlib generator?

Short answer: Clientlib generator automates creation of AEM client libraries from frontend build output.

Asked by: EY, TCS.

4.14 What is Dispatcher configuration?

Short answer: Dispatcher configuration controls caching, filtering, invalidation, and request handling between the web server and AEM publish.

Asked by: Bajaj, EY.

4.15 Difference between static and editable templates.

Short answer: Static templates are developer-controlled and less flexible. Editable templates can be managed by authors using policies and structure mode.

Asked by: Deloitte, Capgemini, Wipro, Bajaj, Accenture.

4.16 What is Parsys and iParsys?

Short answer: Parsys lets authors add components on a page. iParsys allows inherited shared content across pages.

Asked by: Bajaj, Capgemini.

4.17 What is Content Fragment and Experience Fragment?

Short answer: Content Fragment stores structured content, mostly for headless delivery. Experience Fragment stores reusable page/content sections with layout.

Asked by: Bajaj, Deloitte, Techzuf, Wipro, EPAM.

4.18 How do you integrate AEM with React?

Short answer: AEM can expose content through Sling Models, JSON, or GraphQL, and React consumes that data to render UI.

Asked by: Embitel consultancy, Bajaj, Wipro, NTT Data.

4.19 How does React render Content Fragments in AEM?

Short answer: AEM exposes fragment data through GraphQL or JSON APIs, and React fetches and displays it.

Asked by: Bajaj.

4.20 What is MSM in AEM?

Short answer: MSM stands for Multi Site Management. It helps manage multiple sites or country/language copies using blueprints and live copies.

Asked by: Wipro, NTT Data, Accenture, TCS.

4.21 What are run modes in AEM?

Short answer: Run modes define environment-specific behavior such as author, publish, dev, or prod.

Asked by: Wipro, NTT Data.

4.22 What are replication agents?

Short answer: Replication agents are used to move content between Author and Publish environments.

Asked by: Wipro.

4.23 What is Allow Proxy in AEM clientlibs?

Short answer: `allowProxy=true` lets clientlibs under `/apps` be served through `/etc.clientlibs` securely.

Asked by: Concentrix, EPAM.

4.24 What is overlay vs override in AEM?

Short answer: Overlay copies `/libs` structure into `/apps` for customization without changing product code. Override replaces behavior more directly depending on implementation.

Asked by: Accenture, 3 Nov Bajaj.

4.25 How do you create a reusable form component with same functionality but different UI?

Short answer: Keep logic common and make the UI configurable using dialog fields, policies, style system, or conditional rendering.

Asked by: Deloitte.

5. Backend, API, and Security

5.1 How do you connect frontend with backend?

Short answer: Use APIs such as REST, GraphQL, or AEM Servlets, then fetch the data in frontend and render it.

Asked by: Embitel consultancy, IndusInd Bank.

5.2 What is bearer token authentication?

Short answer: The client sends a token in the Authorization header, and the backend validates it before allowing access.

Example:

```
fetch('/api', {
  headers: {
    Authorization: 'Bearer token_here'
  }
});
```

Asked by: Fynd.

5.3 What is authentication vs authorization?

Short answer: Authentication checks who the user is. Authorization checks what the user is allowed to do.

Asked by: Fynd.

5.4 If you have to make 4 API calls, how will you do it and how much time will it take?

Short answer: If they are independent, call them in parallel using `Promise.all`. Total time will be close to the slowest API, not the sum of all four.

Example:

```
const [a, b, c, d] = await Promise.all([
  fetch('/a'),
  fetch('/b'),
  fetch('/c'),
  fetch('/d')
]);
```

Asked by: EY.

6. Practical Coding Questions

6.1 Write a function to find a substring in a string.

```
function hasSubstring(str, sub) {  
  return str.includes(sub);  
}
```

Asked by: Truemeds.

6.2 Find the max number in an array.

```
function maxNum(arr) {  
  return Math.max(...arr);  
}
```

Asked by: Itvedant.

6.3 Remove duplicate numbers from a nested array.

```
function uniqueNested(arr) {  
  return [...new Set(arr.flat(Infinity))];  
}
```

Asked by: SCSS round.

6.4 Return only unique elements and sum them.

```
function uniqueAndSum(arr) {  
  const unique = [...new Set(arr)];  
  const sum = unique.reduce((a, b) => a + b, 0);  
  return { unique, sum };  
}
```

Asked by: Deloitte.

6.5 Flatten a nested object.

```
function flattenObject(obj, prefix = '', result = {}) {  
  for (let key in obj) {  
    const newKey = prefix ? `${prefix}.${key}` : key;  
    if (typeof obj[key] === 'object' && obj[key] !== null) {  
      flattenObject(obj[key], newKey, result);  
    } else {  
      result[newKey] = obj[key];  
    }  
  }  
}
```

```
    return result;
}
```

Asked by: Publicis Sapient.

6.6 Bubble sort without built-in method.

```
function bubbleSort(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length - i - 1; j++) {
      if (arr[j] > arr[j + 1]) {
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }
  return arr;
}
```

Asked by: Wipro.

6.7 Reverse an array.

```
function reverseArray(arr) {
  return arr.reverse();
}
```

Asked by: Embitel consultancy.

6.8 Fetch country list API and then details API on click.

Short answer: First call the list API in `useEffect`, render country buttons/cards, and on click call the details API using the country code or name.

Asked by: Dentsu.

7. Company-wise repetition summary

Company	Repeated topics / focus areas
Bajaj	AEM architecture, editable vs static templates, dispatcher, parsys, clientlibs, Content Fragment, webpack, Author-Publish flow
Truemeds	Lifecycle, useState/useEffect, Redux, middleware, SEO, CSR vs SSR, substring practical
Itvedant	Max array value, accessibility, XHR/fetch, localStorage/sessionStorage/cookies, render API data
Techzuf	Callback, async/await, promises, lifecycle unmount, Content Fragment, Experience Fragment
Fynd	Spread/rest, Promise.allSettled, authorization, useEffect dependency array, accordion, shallow/deep copy
Concentrix	Closures, setTimeout, GraphQL, Adobe Analytics, Adobe Target, templates, clientlibs, allow proxy

Company	Repeated topics / focus areas
Deloitte	Template, component, style guide, clientlibs, Content Fragment, Experience Fragment, reusable form component, style policies
Capgemini	OSGi, JCR, annotations, libs/apps, parsys/iParsys, clientlibs, embed vs dependencies, editable vs static templates
Publicis Sapient	PWA, tree shaking, webpack, Redux side effects, middleware, controlled vs uncontrolled, closures, useMemo/useCallback/React.memo
IndusInd Bank	Annotations, Sling Model, OSGi, backend/frontend integration, encryption, pass data to backend
Morgan Stanley	AEM basics, project architecture, stakeholders, collaboration, content publishing, deep links
EPAM	HTML4 vs HTML5, meta tags, Flex/Grid, CSS/SCSS, event bubbling, allow proxy, clientlibs, data-sly directives
Wipro	Accessibility, SEO, closures, promises, async/await, workers, templates, MSM, run modes, replication agents, bubble sort
HCL	AEM, Jest, state management, Redux vs Context, Virtual DOM, reconciliation, keys, hooks, controlled input
Movate	React and AEM basics, practical questions
NTT Data	Accordion in AEM, AEM to React rendering, Sling/HTL model data, run modes, Jenkins, rewrite rules, MSM, React Router, chunks
UST Global	CMS migration, reusable component libraries, multi-site architecture, performance, multi-country handling
Accenture	Flex/Grid, negative margin, JS execution order, Author vs Publish, overlay vs override, MSM, templates
TCS	AEM basics, page policy, component policy, CSS not loading debug, clientlib generator, GraphQL, MSM, multilingual error handling
EY	Flexbox/Grid, media queries, CSS scenarios, banner layout, responsive images in HTML, Sling Model annotations, HTL consumption, servlet, clientlib generator, embed vs dependencies, dispatcher configuration, ES6, arrow functions, promises, event loop, parallel API calls

8. Most repeated questions overall

Topic	Approx. count
AEM architecture	9
Static vs editable template	8
Clientlibs	8
Hooks / useState / useEffect	7
Closures	7
let / const / var	7
Promises / async-await	7
Flexbox vs Grid	7
Dispatcher	6

Topic	Approx. count
Content Fragment / Experience Fragment	6
Redux vs Context API	6
Performance optimization	6
Event loop / setTimeout	6
Sling Model	5
MSM	5
Controlled vs uncontrolled components	5
Virtual DOM / Reconciliation	5
Webpack / bundles / chunks	5

9. Final revision order

1. AEM architecture, templates, clientlibs, dispatcher, Sling Models.
2. React hooks, state, routing, Redux vs Context, optimization.
3. JavaScript fundamentals, promises, event loop, closures.
4. HTML/CSS scenarios and practicals.
5. AEM + React integration, GraphQL, MSM.
6. Practical coding questions.
7. Project explanation, current role, stakeholders, and communication answers.